ODMM V228.47 - Analyse du cout SQLite

Auteur

Abdelhalim Talbi

Objet

Caracteriser les sous-operations SQLite du moteur IncrementalExecutor, sans modifier les garanties de securite.

Methode

Benchmark du chemin reel local, cinq repetitions par taille, inventaire et reservation exclus du chronometrage.

Resultats

4096 octets : total 3.6126 ms ; SQLite 0.6002 ms ; part mediane 13.8 %

65536 octets : total 3.8984 ms ; SQLite 0.5813 ms ; part mediane 14.97 %

1048576 octets : total 7.0008 ms ; SQLite 0.5206 ms ; part mediane 7.44 %

Architecture

Source POSIX -> NativePosixMigration -> IncrementalExecutor -> controles SHA-256 -> transaction SQLite -> validation. Les operations de lecture et de commit restent sous fencing.

Organigramme

Reserver -> Copier -> Verifier -> BEGIN IMMEDIATE -> Revérifier -> COMMIT si valide, sinon ROLLBACK et reconciliation explicite.

Qualification

18 tests cibles reussis. Aucune migration NAS reelle, compilation Windows native ni test ACL SMB/NFS qualifie.

Limites

Mesures locales et caches non maitrises. Le cout SQLite presente ne couvre que les sous-etapes instrumentees. Aucun gain de performance revendique.

Exemple fictif clairement identifie

Commande illustrative (non executee sur NAS): python sqlite_cost_analysis_v22847.py benchmark.json --output result.json

Decision

Ne pas regrouper les COMMIT sans preuve de reprise, idempotence et fencing. Prochaine etape: prototype isole et comparatif A/B.